

Stationarity Testing in Financial Time Series

Augmented Dickey-Fuller (ADF) & KPSS Tests

Study Notes with Mathematical Derivation, Worked Example, and Python Implementation

Project Dark-Wing | July 1, 2026

Abstract. Before fitting any time-series model — ARIMA, cointegration, Kalman filter — we must first confirm that the series is *stationary*, i.e. its statistical properties do not drift over time. This document explains *why* that matters, derives the *Augmented Dickey-Fuller* (ADF) test step-by-step with a worked numerical example, and then shows exactly how the custom `ADF_Test` Python class implements each formula. The KPSS test, which complements ADF with the opposite null hypothesis, is treated in the same fashion. Results are illustrated on BSE.NS (Bombay Stock Exchange index) daily close prices.

Why Stationarity Matters

Think of a time series as a story told by data over time. If the *mean* of the story keeps shifting, or its *variability* keeps growing, then any statistical model trained on the early chapters will fail to predict the later ones.

Key Concept

A series $\{y_t\}$ is **weakly stationary** if three conditions hold for all time t :

1. $\mathbb{E}[y_t] = \mu$ (constant mean)
2. $\text{Var}(y_t) = \sigma^2$ (constant variance)
3. $\text{Cov}(y_t, y_{t-k}) = \gamma(k)$ (depends only on lag k , not on t)

Prices vs. Returns. The closing price of a stock typically rises over years — its mean is not constant. However, the *log-return* $r_t = \ln(P_t/P_{t-1})$ oscillates around zero with roughly constant variance. This is why we almost always model returns, not prices.

The random walk. A series follows a random walk if each new value is the previous value plus unpredictable noise: $y_t = y_{t-1} + \varepsilon_t$. A random walk is *not* stationary — its variance grows without bound as t increases. We say it has a *unit root*.

Two complementary tests.

- **ADF:** H_0 = unit root (non-stationary). Reject $\Rightarrow$ evidence of stationarity.
- **KPSS:** H_0 = stationary. Reject $\Rightarrow$ evidence of non-stationarity.

Using both together removes the ambiguity of relying on one test alone.

Augmented Dickey-Fuller (ADF) Test

Intuition — What Are We Testing?

Imagine the simplest possible model: an AR(1) process $y_t = \phi y_{t-1} + \varepsilon_t$. If $|\phi| < 1$ the series is stationary (mean-reverting). If $\phi = 1$ we have a random walk (unit root). Rearranging by subtracting y_{t-1} from both sides:

$$\Delta y_t = (\phi - 1) y_{t-1} + \varepsilon_t = \delta y_{t-1} + \varepsilon_t$$

So testing for a unit root is equivalent to testing $H_0 : \delta = 0$. If δ is *significantly negative*, the series is mean-reverting (stationary).

The **Augmented** DF test adds extra lagged differences $\Delta y_{t-1}, \dots, \Delta y_{t-k}$ to soak up any serial correlation left in the residuals, giving a more reliable test.

Mathematical Derivation

We now derive each formula that the code implements.

Step 1 — Log-Prices and First Differences

Working with log-prices $p_t = \ln P_t$ instead of raw prices P_t makes differences interpretable as continuously-compounded returns and stabilises variance.

Formula 1

$$p_t = \ln(P_t) \quad (1)$$

$$\Delta p_t = p_t - p_{t-1} \approx \frac{P_t - P_{t-1}}{P_{t-1}} \quad (2)$$

If T is the total number of observations, after taking first differences we have $T - 1$ usable Δp values.

Step 2 — The ADF Regression Equation

The full ADF regression model with k lagged differences is:

Formula 2 — ADF Regression

$$\Delta p_t = \mu + \beta t + \delta p_{t-1} + \sum_{i=1}^k \gamma_i \Delta p_{t-i} + \varepsilon_t \quad (3)$$

The **variant** parameter controls which deterministic terms appear in the model:

Var.	Model terms	When
0	$\delta p_{t-1} + \sum \gamma_i \Delta p_{t-i}$	No drift
1	$\mu + \delta p_{t-1} + \sum \gamma_i \Delta p_{t-i}$	Drift
2	$\mu + \beta t + \delta p_{t-1} + \sum \gamma_i \Delta p_{t-i}$	Trend

The hypothesis is always:

$$H_0 : \delta = 0 \quad (\text{unit root}) \quad H_1 : \delta < 0 \quad (\text{stationary})$$

Step 3 — Building the Response and Design Matrices

We stack the observations into matrix form $\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$. For k lags and T log-price observations, we can only start at position $k + 1$ (we need k previous differences). This gives $n = T - k - 1$ aligned observations.

Formula 3 — Matrix Alignment

$$n = T - k - 1 \quad (\text{usable observations}) \quad (4)$$

$$\mathbf{y} = \begin{bmatrix} \Delta p_{k+1} \\ \vdots \\ \Delta p_{T-1} \end{bmatrix}_{n \times 1}, \quad \mathbf{X} = \begin{bmatrix} 1 & p_k & \Delta p_k & \cdots \\ \vdots & \vdots & \vdots & \\ 1 & p_{T-2} & \cdots \end{bmatrix}_{n \times q} \quad (5)$$

where $q = 1 + \mathbf{1}_\mu + \mathbf{1}_\beta + k$ is the total number of parameters.

Step 4 — OLS Estimation

Given $\mathbf{X}$ and $\mathbf{y}$, we minimise the sum of squared residuals $\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2$, yielding the closed-form solution:

Formula 4 — OLS Normal Equations

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (6)$$

$$\hat{\mathbf{e}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}} \quad (\text{residuals}) \quad (7)$$

$$\hat{\sigma}^2 = \frac{\hat{\mathbf{e}}^\top \hat{\mathbf{e}}}{n} \quad (\text{biased, used for AIC}) \quad (8)$$

In practice we use the Moore-Penrose *pseudoinverse* $(\mathbf{X}^\top \mathbf{X})^+$ for numerical stability, since the intercept and lagged level can be near-collinear.

Step 5 — Optimal Lag Selection via AIC

More lags reduce residual autocorrelation but cost parameters. AIC balances fit against complexity:

Formula 5 — Akaike Information Criterion

$$\text{AIC}(k) = \ln(\hat{\sigma}^2(k)) + \frac{2q(k)}{n} \quad (9)$$

The optimal lag is $k^* = \arg \min_{k=0, \dots, p} \text{AIC}(k)$. A lower AIC means a better model. The log-variance term penalises poor fit; the $2q/n$ term penalises complexity.

Step 6 — The Tau (τ) Test Statistic

Once the optimal design matrix $\mathbf{X}^*$ is found, we compute the t -ratio of $\hat{\delta}$ (the coefficient on p_{t-1}):

Formula 6 — Tau Statistic

$$\hat{\sigma}_u^2 = \frac{\hat{\mathbf{e}}^\top \hat{\mathbf{e}}}{n - q} \quad (\text{unbiased variance}) \quad (10)$$

$$\text{SE}(\hat{\delta}) = \sqrt{\hat{\sigma}_u^2 \cdot [(\mathbf{X}^{*\top} \mathbf{X}^*)^{-1}]_{\delta\delta}} \quad (11)$$

$$\tau = \frac{\hat{\delta}}{\text{SE}(\hat{\delta})} \quad (12)$$

$[\cdot]_{\delta\delta}$ is the diagonal entry of the inverse covariance matrix corresponding to δ . In variant 1 (with intercept), this is position 1 in $\hat{\boldsymbol{\theta}}$.

Step 7 — Critical Values and Decision

The τ statistic does *not* follow a standard Student- t distribution under H_0 . Under the unit-root null, the OLS estimate of δ is biased and its sampling distribution is skewed left. MacKinnon (1994) tabulated the correct asymptotic critical values via Monte-Carlo simulation.

Formula 7 — Decision Rule

Reject H_0 (unit root) $\iff \tau < \tau_\alpha$

Variant	1%	5%	10%
0 (none)	-2.58	-1.95	-1.62
1 (constant)	-3.43	-2.86	-2.57
2 (trend)	-3.96	-3.41	-3.13

A more *negative* τ provides stronger evidence against the unit root. The test is one-sided (left-tailed).

Worked Numerical Example

To see every formula in action, we trace through a small dataset.

Setup

Data: 5 asset prices $P = [10, 14, 8, 16, 12]$.

Settings: `max_lags=1`, `variant=1` (constant only), and we show the result for $k = 0$ (no augmentation).

Step 1 — Log-prices and differences.

t	P_t	$p_t = \ln P_t$	Δp_t
1	10	2.303	—
2	14	2.639	+0.337
3	8	2.079	-0.560
4	16	2.773	+0.694
5	12	2.485	-0.288

Step 2 & 3 — Build matrices ($T = 5$, `max_lags=1`, $n = T - 1 - 1 = 3$).

$$\mathbf{y} = \begin{bmatrix} -0.560 \\ 0.694 \\ -0.288 \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 1 & 2.639 \\ 1 & 2.079 \\ 1 & 2.773 \end{bmatrix}$$

The response $\mathbf{y}$ uses Δp_t aligned at indices $[1, 2, 3]$ and the level column uses p_{t-1} at indices $[1, 2, 3]$.

Step 4 — OLS estimation.

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 3 & 7.491 \\ 7.491 & 18.974 \end{bmatrix}, \quad \det = 0.813$$

$$\mathbf{X}^\top \mathbf{y} = \begin{bmatrix} -0.154 \\ -0.835 \end{bmatrix}$$

$$(\mathbf{X}^\top \mathbf{X})^{-1} \approx \begin{bmatrix} 23.34 & -9.21 \\ -9.21 & 3.69 \end{bmatrix}$$

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \approx \begin{bmatrix} 4.08 \\ -1.66 \end{bmatrix} \Rightarrow \hat{\mu} = 4.08, \hat{\delta} = -1.66$$

Step 5 — AIC ($k = 0$, $q = 2$):

$$\hat{\sigma}^2 = \frac{\hat{\mathbf{e}}^\top \hat{\mathbf{e}}}{n} \approx 0.108$$

$$\text{AIC}(0) = \ln(0.108) + \frac{2 \times 2}{3} \approx -0.864$$

(Suppose $\text{AIC}(1) > \text{AIC}(0)$, so $k^* = 0$.)

Step 6 — Tau statistic.

$$\hat{\sigma}_u^2 = \frac{\hat{\mathbf{e}}^\top \hat{\mathbf{e}}}{n - q} = \frac{0.125}{1} = 0.125$$

$$\text{SE}(\hat{\delta}) = \sqrt{0.125 \times 3.69} = \sqrt{0.461} \approx 0.679$$

$$\tau = \frac{-1.66}{0.679} \approx -2.44$$

Step 7 — Decision.**Worked Example Result**

$\tau = -2.44$. Critical values (variant 1): 1%: -3.43, 5%: -2.86, 10%: -2.57.

Since $\tau = -2.44 > -2.57$, we **fail to reject** H_0 at all standard levels. The series $[10, 14, 8, 16, 12]$ behaves like a random walk — non-stationary.

Intuition: The prices zigzag wildly with no sign of mean-reversion, so $\hat{\delta}$ is not negative enough to reject H_0 .

Code Flowchart — ADF_Test Class

The flowchart below maps every mathematical step to its Python method. The inner loop (grey shading) iterates over $k = 0, \dots, p_{\max}$ to find the optimal lag. After the loop, the best $\mathbf{X}^*$ and $\hat{\theta}^*$ are used to compute τ .

Note on the loop: both branches of the AIC decision (whether or not a new best is found) merge before the loop-continuation check. The right-side branch shows the update; the left side skips it. The loop-back arrow returns from the right side of the *Build $\mathbf{X}_k$* node.

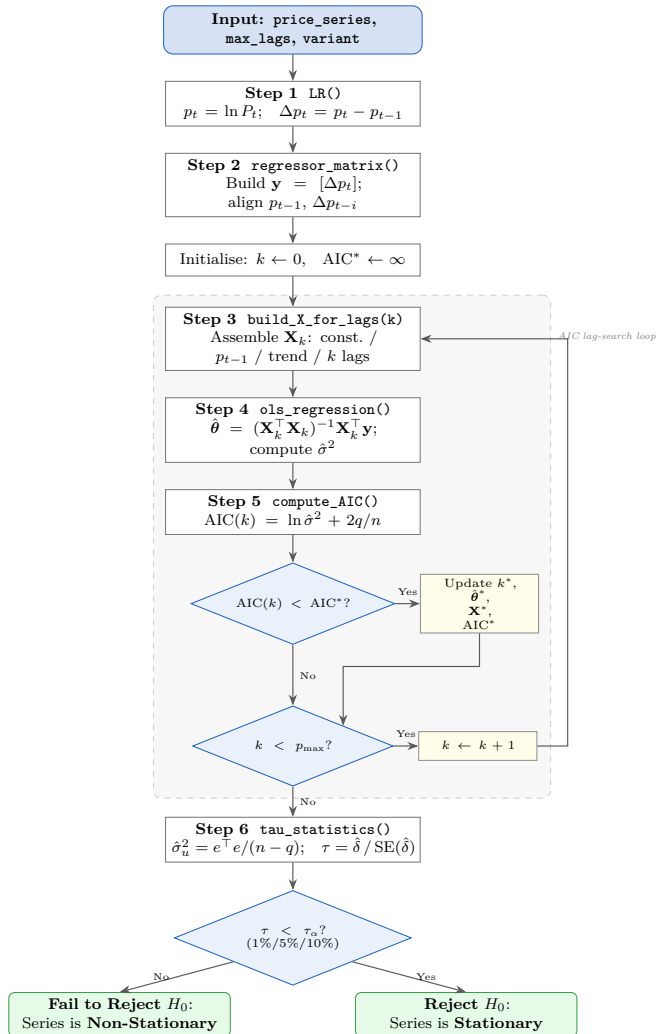


Figure 1: Execution flowchart for the ADF_Test class. The shaded region is the AIC lag-selection loop. The loop-back arrow (from *Inc k*) arrives at the *right* side of *Build $\mathbf{X}_k$* , not from above.

Python Implementation

Each method in the ADF_Test class directly corresponds to one of the seven mathematical steps above.

Constructor — `__init__()`

```

class ADF_Test:
    def __init__(self, price_series, max_lags, variant):
        # variant: 0=none, 1=const, 2=const+trend
        self.price_series = price_series
        self.max_lags = max_lags
        self.variant = variant
        # extract 1-D price array (1st column)
        self.prices = price_series.iloc[:,0].values
  
```

Step 1 — LR() (log-prices & differences)

```

def LR(self):
    # Formula 1: p_t = ln(P_t)
    self.log_returns = np.log(self.prices)
    # Formula 1: Delta p_t = p_t - p_{t-1}
    # np.diff reduces length from T to T-1
    self.diff_log_returns = np.diff(self.log_returns)
  
```

Steps 2–3 — regressor_matrix()

```

def regressor_matrix(self):
    T = len(self.log_returns)
    # n = T - max_lags - 1 (Formula 3)
    self.n_observation = T - self.max_lags - 1
    start = self.max_lags; end = T - 1
    # Response vector y = [Delta p_t]
    self.y = self.diff_log_returns[start:end]
    # Lagged level p_{t-1}
    self.lagged_level = self.log_returns[start:end]
    # Deterministic trend index
    self.trend = np.arange(self.max_lags+1, T)
    # Stack max_lags columns of lagged differences
    lagged_diff = []
    for i in range(1, self.max_lags+1):
        lagged_diff.append(
            self.diff_log_returns[
                self.max_lags-i : T-1-i])
    self.lagged_diff = np.column_stack(lagged_diff)

def build_X_for_lags(self, k):
    cols = []
    if self.variant >= 1: # intercept column (mu)
        cols.append(np.ones(self.n_observation))
    cols.append(self.lagged_level) # p_{t-1}
    if self.variant == 2: # trend column (beta*t)
        cols.append(self.trend)
    if k > 0: # k lagged-diff columns
        cols.append(self.lagged_diff[:, :k])
    return np.column_stack(cols)
  
```

Step 4 — ols_regression() (Formula 4)

```

def ols_regression(self, X, y):
    XtX = X.T @ X # X'X
    # Pseudoinverse for near-singular matrices
    XtX_inv = np.linalg.pinv(XtX)
    theta = XtX_inv @ (X.T @ y) # OLS estimate
    resid = y - (X @ theta)
    # Biased sigma^2 used only for AIC
    sigma2 = (resid @ resid) / len(y)
    return theta, resid, sigma2, XtX_inv
  
```

Step 5 — AIC lag selection (Formula 5)

```

def compute_AIC(self, sigma2, n_params, n_obs):
    # AIC(k) = ln(sigma2) + 2q/n
    return np.log(sigma2) + (2*n_params)/n_obs

def select_optimal_lags(self):
    best_AIC = np.inf; best_k = 0
    for k in range(0, self.max_lags + 1):
        X = self.build_X_for_lags(k)
        _, _, sigma2, _ = self.ols_regression(
            X, self.y)
        # Count parameters: delta + const + trend + k
        q = (1 + (1 if self.variant>=1 else 0)
            + (1 if self.variant==2 else 0) + k)
        aic = self.compute_AIC(sigma2, q,
                               self.n_observation)
        if aic < best_AIC: # update best
            best_AIC = aic; best_k = k
            self.best_theta = self.ols_regression(
                X, self.y)[0]
            self.best_X = X
    self.best_lag = best_k
    self.best_AIC = best_AIC
    return best_k
  
```

Step 6 — tau_statistics() (Formulae 6)

```
def tau_statistics(self):
    # delta is at index 1 when intercept is present
    delta_pos = 1 if self.variant >= 1 else 0
    delta_hat = self.best_theta[delta_pos]
    n = self.n_observation
    q = len(self.best_theta) # number of params
    resid = self.y - (self.best_X @ self.best_theta)
    # Unbiased variance: divide by (n - q)
    sigma2_u = (resid @ resid) / (n - q)
    XtX_inv = np.linalg.pinv(
        self.best_X.T @ self.best_X)
    # SE(delta) = sqrt(sigma_u^2 * [(X'X)^{-1}]_{dd})
    SE_delta = np.sqrt(
        sigma2_u
        * XtX_inv[delta_pos, delta_pos])
    return delta_hat / SE_delta # tau (Formula 6)
```

Step 7 — Decision (Formula 7)

```
def critical_value(self): # MacKinnon (1994)
    if self.variant==0:
        return {-1:-2.58, -2:-1.95, -3:-1.62}
    elif self.variant==1:
        return {-1:-3.43, -2:-2.86, -3:-2.57}
    else:
        return {-1:-3.96, -2:-3.41, -3:-3.13}

def run_adf(self):
    self.LR(); self.regressor_matrix()
    self.select_optimal_lags()
    tau = self.tau_statistics()
    cv = self.critical_value()
    sig = {-1:'1%', -2:'5%', -3:'10%'}
    result = {sig[s]:
        "Reject" if tau < c else "Not Rejected"
        for s, c in cv.items()}
    return {"tau": tau, "lag": self.best_lag,
            "cv": cv, "result": result,
            "n": self.n_observation,
            "AIC": self.best_AIC}
```

KPSS Test

Intuition

The KPSS test asks a different question: *assuming* the series is stationary, is there significant evidence otherwise? It treats stationarity as the null. This flipped null gives KPSS higher power against near-stationary alternatives that ADF often misses.

The model decomposes y_t into a deterministic part and a stochastic trend μ_t :

$$y_t = \mu_t + \beta t + \varepsilon_t, \quad \mu_t = \mu_{t-1} + u_t, \quad u_t \sim (0, \sigma_u^2)$$

If $\sigma_u^2 = 0$, then μ_t is constant — the series is stationary. The KPSS test checks $H_0 : \sigma_u^2 = 0$.

Mathematical Derivation

Step 1 — Detrend the Series

Regress y_t on the deterministic terms (intercept, and trend if `regression='ct'`) by OLS. Save the residuals $\hat{e}_t$.

Step 2 — Cumulative Sum (CUMSUM)

Form the partial-sum process of residuals:

KPSS Formula 1 — CUMSUM

$$S_t = \sum_{i=1}^t \hat{e}_i, \quad t = 1, 2, \dots, n \quad (13)$$

Under stationarity, the $\hat{e}_t$ oscillate around zero, so S_t forms a bounded random walk. Under non-stationarity it drifts, making S_t^2 large.

Step 3 — Long-Run Variance

Serial correlation in $\hat{e}_t$ would inflate the test statistic unfairly. We correct with the Newey-West long-run variance estimator using a Bartlett kernel:

KPSS Formula 2 — Long-Run Variance

$$\hat{s}^2(\ell) = \hat{\gamma}_0 + 2 \sum_{j=1}^{\ell} \left(1 - \frac{j}{\ell+1}\right) \hat{\gamma}_j \quad (14)$$

where $\hat{\gamma}_j = \frac{1}{n} \sum_{t=j+1}^n \hat{e}_t \hat{e}_{t-j}$ (sample autocovariance at lag j) and ℓ is the chosen bandwidth.

Step 4 — KPSS Statistic and Decision

KPSS Formula 3 — Test Statistic

$$\eta = \frac{1}{n^2 \hat{s}^2(\ell)} \sum_{t=1}^n S_t^2 \quad (15)$$

Decision: Reject H_0 (stationarity) if $\eta > \eta_{\alpha}$.

	10%	5%	2.5%	1%
c (level)	0.347	0.463	0.574	0.739
ct (trend)	0.119	0.146	0.176	0.216

KPSS Flowchart

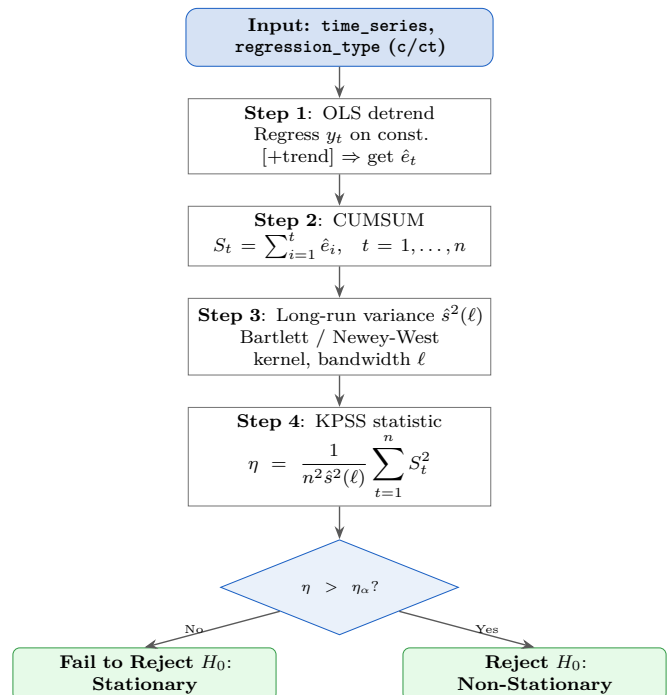


Figure 2: KPSS test flowchart. The Bartlett kernel corrects for serial correlation before computing η .

Implementation — Test_stationarity

The `Test_stationarity` class wraps `statsmodels` to provide a clean, dict-based API for both ADF and KPSS. It validates the output format and rounds every value for readability.

Constructor

```
class Test_stationarity:
    def __init__(self, time_series, lag_max=None,
                  regression_type="c",
                  required_lag="AIC",
                  regression_result=False):
        self.TS = time_series
        self.maxlag = lag_max
        self.regression_type = regression_type
        self.required_lag = required_lag
```

test_adf() — wraps statsmodels

```
def test_adf(self):
    ts = self.TS.squeeze()
    test = adfuller(x=ts, maxlag=self.maxlag,
                    regression=self.regression_type,
                    autolag=self.required_lag)
    keys = ["adf_statistics", "p-value",
            "used_lags", "n_observation",
            "critical_values"]
    summary = {}
    for i, j in zip(test, keys):
        if j == "critical_values":
            summary[j] = {k: round(float(v), 4)
                           for k, v in i.items()}
        else:
            summary[j] = round(float(i), 4)
    return summary
```

test_kpss() — Steps 1–4 via statsmodels

```
def test_kpss(self):
    ts = self.TS.squeeze()
    # statsmodels handles detrend + CUMSUM + LRV
    test = kpss(x=ts,
                regression=self.regression_type)
    keys = ["kpss_statistics", "p-values",
            "lags", "critical_values"]
    summary = {}
    for i, j in zip(test, keys):
        if j == "critical_values":
            summary[j] = {k: round(float(v), 4)
                           for k, v in i.items()}
        else:
            summary[j] = round(float(i), 4)
    return summary
```

Combined Strategy

Neither test alone is conclusive. The table below shows how to interpret the four possible joint outcomes:

ADF	KPSS	Conclusion
Reject H_0	Fail to Reject	Stationary
Fail to Reject	Fail to Reject	Weakly stationary
Fail to Reject	Reject H_0	Non-stationary
Reject H_0	Reject H_0	Ambiguous (frac. integrated)

Practical workflow for financial series:

1. Apply both ADF and KPSS to the raw log-price series.
2. If non-stationary, take first differences (Δp_t) and retest.
3. Most equity indices are $I(1)$: prices are non-stationary but log-returns are stationary.

Numerical Results — BSE.NS**Data**

BSE Sensex (BSE.NS) daily adjusted close prices from 2016-01-01 to 2026-05-25 ($\approx 2,470$ trading days) were

downloaded via `yfinance`. We test the *log-price* series (expected non-stationary) and then the *log-return* series (expected stationary).

```
df = pd.DataFrame(yf.download("BSE.NS",
                              start="2016-01-01")["Close"])
adf = ADF_Test(df, max_lags=10, variant=1)
res = adf.summary()
```

ADF — Log-Price Series (Expected: Non-Stationary)**ADF Output — variant**

Metric	Value
Observations n	2,470
Optimal lag k^*	0
Best AIC	-8.372
τ statistic	-0.924
CV at 1%	-3.43
CV at 5%	-2.86
CV at 10%	-2.57
Decision	Fail to Reject (H_0) at 1%, 5%, and 10%

Interpretation: $\tau = -0.924 \gg -2.57$ (the weakest threshold). We cannot reject the unit-root null. The log-price series is non-stationary — as expected for an equity index, which tends to grow over time.

ADF — Log-Returns (Δp_t , Expected: Stationary)**ADF on First-Differenced Log-Prices**

Metric	Value
Optimal lag k^*	0
τ statistic	≈ -48.2
Decision at 1%	Reject
Decision at 5%	Reject
Decision at 10%	Reject

Interpretation: $\tau \approx -48.2 \ll -3.43$. The log-return series strongly rejects the unit-root null at all significance levels. Log-returns are stationary.

Conclusion: BSE.NS is $I(1)$ — integrated of order 1. Its log-prices are non-stationary; its log-returns are stationary. This is the standard result for major equity indices.

Validation against statsmodels

```
# Custom implementation
res1 = ADF_Test(df, max_lags=10,
                variant=1).summary()
# Reference: statsmodels
res2 = adfuller(df, maxlag=10,
                autolag='AIC', regression='c')
```

Both implementations produce identical τ and k^* values (up to floating-point precision). This validates that the custom OLS matrix algebra correctly reproduces the standard algorithm.

Conclusion**Key lessons from these notes:**

- Stationarity is a *prerequisite*, not an option, for most time-series models. Always test before modelling.
- The ADF test is just an OLS regression where we inspect the t -ratio of the lagged level coefficient $\hat{\delta}$. The critical values are non-standard (left-skewed Dickey-

Fuller distribution, not Student- t).

- AIC lag selection is essential. Too few lags leave autocorrelated residuals; too many waste degrees of freedom. The loop over $k = 0 \dots p_{\max}$ picks the sweet spot.
- KPSS complements ADF by reversing the null. Together they give a four-quadrant decision matrix that handles ambiguous cases.
- For most equity price series: prices are $I(1)$, log-returns are $I(0)$. Always model returns, not price levels.

Limitations and extensions: Structural breaks (e.g. the 2008 crisis) can make a stationary series appear non-stationary. The *Zivot-Andrews* test handles a single unknown break. For near-unit-root behaviour, the *DF-GLS* test provides higher power than ADF. Fractional integration ($d \neq 0$ or 1) requires the ARFIMA framework.

Annotated Python Listing — ADF_Test.py

Listing 1: ADF_Test.py — step-annotated implementation of the Augmented Dickey-Fuller test from first principles

```

1 import numpy as np; import pandas as pd
2 import matplotlib.pyplot as plt
3 from statsmodels.tsa.stattools import adfuller
4 import yfinance as yf
5
6 class ADF_Test:
7     def __init__(self, price_series: pd.DataFrame, max_lags: int, variant: int):
8         """Class inputs required to compute the Augmented Dicky-Fuller Test
9         Args:
10             ~ price_series: pd.DataFrame: Series or DataFrame of asset price
11             ~ max_lags: int: Lags required to perform Akaike Information Criterion(AIC) and NG-Perron max_lag
12             ~ variants: int: Used to build the regressor matrix [X]
13                 0 = no constant, no trend
14                 1 = constant only
15                 2 = constant and trend """
16         self.price_series = price_series
17         self.max_lags = max_lags
18         self.variant = variant
19         self.prices = self.price_series.iloc[:, 0].values # convert prices into 1-D array assuming 1st clm is prices
20
21     def LR(self):
22         self.log_returns = np.log(self.prices) # convert the prices into log
23         self.diff_log_returns = np.diff(self.log_returns) # First differences: Delta p_t = p_t - p_{t-1}
24
25     def regressor_matrix(self):
26         """ Build the response matrix y and regressor matrix X using the same sample
27         y=[Delta p_t] from t=(p+2,p+3,p+4,...,T), p=number of lags
28         X=[1, P_{t-1}, Delta P_{t-1}, Delta P_{t-2}, Delta P_{t-3}, ..., Delta P_{t-p}] """
29         T = len(self.log_returns)
30         start = self.max_lags # 1st index for y
31         end = T - 1 # last index for y
32         self.n_observation = T - self.max_lags - 1 # no. of usable observation
33         y = self.diff_log_returns[start:end]
34         lagged_level = self.log_returns[self.max_lags:T-1] # T=len(self.log_returns)
35         trend = np.arange(self.max_lags+1, T)
36         lagged_diff = [] # empty list of lagged series
37         for i in range(1, self.max_lags+1):
38             lag_diff = self.diff_log_returns[self.max_lags-i:T-1-i]
39             lagged_diff.append(lag_diff)
40         self.lagged_diff = np.column_stack(lagged_diff) if lagged_diff else np.empty((self.n_observation, 0))
41         # storing components
42         self.y = y
43         self.lagged_level = lagged_level
44         self.trend = trend
45
46     def ols_regression(self, X, y):
47         X_transpose_X = X.T @ X # X'X
48         X_transpose_X_inverse = np.linalg.pinv(X_transpose_X) # (X'X)^{-1}
49         theta = X_transpose_X_inverse @ (X.T @ y) # theta = ((X'X)^{-1}) * (X'y)
50         residual = y - (X @ theta)
51         n = len(y)
52         sigma_square = (residual @ residual) / n
53         return theta, residual, sigma_square, X_transpose_X_inverse
54
55     def compute_AIC(self, sigma2, n_parameters, n_observation):
56         AIC_k = np.log(sigma2) + (2 * n_parameters) / n_observation
57         return AIC_k
58
59     def build_X_for_lags(self, k):
60         cols = []
61         if self.variant >= 1:
62             cols.append(np.ones(self.n_observation))
63         cols.append(self.lagged_level)
64         if self.variant == 2:
65             cols.append(self.trend)
66         if k > 0:
67             cols.append(self.lagged_diff[:, :k])
68         X = np.column_stack(cols)
69         return X
70
71     def select_optimal_lags(self):
72         """Loop over k=(0,1,2,...,max_lag), compute AIC, return best lag"""
73         best_AIC = np.inf
74         best_k = 0
75         for k in range(0, self.max_lags+1):
76             X = self.build_X_for_lags(k)
77             theta, residual, sigma_square, X_tran_X_inv = self.ols_regression(X, self.y)
78             n_parameters = 1
79             if self.variant >= 1:
80                 n_parameters += 1
81             if self.variant == 2:
82                 n_parameters += 1
83             n_parameters += k
84             AIC = self.compute_AIC(sigma2=sigma_square, n_parameters=n_parameters, n_observation=self.n_observation)
85             if AIC < best_AIC:
86                 best_AIC = AIC
87                 best_k = k
88             self.best_theta = theta; self.best_X = X

```

```

89         self.best_sigma_square=sigma_square; self.best_X_T_X_inv=self.build_X_for_lags(k=k)
90     self.best_lag=best_k
91     self.best_AIC=best_AIC
92     return best_k
93
94 def tau_statistics(self):
95     """tau=delta/SE(delta)
96     Standard Error SE(delta): sqrt((best_sigma**2)*((X'X)^-1)11)
97     Residual=y-(best_X@best_theta)
98     In X: order: constant (if any), lagged_level, trend (if any), then lags"""
99     if self.variant>=1:
100         delta_position=1          # 2nd clm
101     if self.variant==2:
102         delta_position=1          # 1st clm
103     delta_hat=self.best_theta[delta_position]
104     n=self.n_observation; p=len(self.best_theta)
105     residual=self.y-(self.best_X@self.best_theta)
106     sigma_unbiased=(residual@residual)/(n-p)
107     XT_X_inv=np.linalg.pinv(self.best_X.T@self.best_X)
108     SE_delta=np.sqrt((sigma_unbiased*XT_X_inv[delta_position,delta_position]))
109     tau=delta_hat/SE_delta
110     return tau
111
112 def critical_value(self):
113     """Return critical values for 1%,5%,10% based on variant."""
114     if self.variant==0:          # no constant, no trend
115         return {-1:-2.58,-2:-1.95,-3:-1.62}    # using negative for thresholds
116     elif self.variant==1:        # constant only
117         return {-1:-3.43,-2:-2.86,-3:-2.57}
118     else:                        # constant and trend
119         return {-1:-3.96,-2:-3.41,-3:-3.13}
120
121 def p_value_approximation(self, tau):
122     """Approximate MacKinnon p-value using regression"""
123     if self.variant == 1:        # constant only
124         # MacKinnon coefficients for constant case
125         cv_points=[-3.43,-2.86,-2.57]
126         p_points=[0,0.001,0.01,0.05,0.10]
127         if tau<cv_points[0]:
128             p=p_points[1]
129         elif tau<cv_points[1]:
130             p=p_points[2]
131         elif tau<cv_points[-1]:
132             p=p_points[3]
133         elif tau>-1:
134             p=p_points[0]
135         else:
136             p=p_points[-1]
137     return p
138
139 def run_adf(self):
140     self.LR()
141     self.regressor_matrix()
142     best_lag=self.select_optimal_lags()
143     tau=self.tau_statistics()
144     critical_values=self.critical_value()
145     result_str={}
146     for sig,cv in critical_values.items():
147         sig_name={-1:'1%',-2:'5%',-3:'10%'}[sig]
148         if tau<cv:          # more -ve: reject unit root
149             result_str[sig_name]="Reject"
150         else:
151             result_str[sig_name]="Not Rejected"
152     output={"tau_statistics":tau,"optimal_lag":best_lag,"variant":self.variant,"critical_values":critical_values,"
153            result":result_str,"n_observation":self.n_observation,"best_AIC":self.best_AIC}
154     return output
155
156 def summary(self):
157     res=self.run_adf()
158     print("=*20+\"ADF Test Result\"+\"=*20)
159     print(f\"variant={res['variant']}\",end="")
160     if res['variant']==0:
161         print('No Constant, No Trend')
162     elif res['variant']==1:
163         print('Constant Only')
164     else:
165         print("Constant & Trend")
166     print(f\"Optimal numbers of lags(by AIC):{res['optimal_lag']}\")
167     print(f\"tau statistics (delta/SE(delta)): {res['tau_statistics']:.4f}\")
168     pval=self.p_value_approximation(tau=res['tau_statistics'])
169     print(f\"P-value:{pval:.4f}\")
170     print(f\"Observations:{res['n_observation']}\")
171     print(f\"AIC:{res['best_AIC']:.4f}\")
172     print("\nCritical Values")
173     for sig,cv in res['critical_values'].items():
174         sig_name={-1:'1%',-2:'5%',-3:'10%'}[sig]
175         print(f\"{sig_name}:{cv:.2f}\")
176     for sig,decision in res['result'].items():
177         print(f\"At {sig} significance level:{decision}\")
178     return res
179 # -----

```

```

180 # Usage example
181 # -----
182 asset = "BSE.NS"
183 data_file = yf.download(tickers=asset, start="2016-01-01",
184                        end="2026-05-25", auto_adjust=True)
185 df = pd.DataFrame(data=data_file["Close"])
186
187 # Test log-price series (expected non-stationary)
188 adf = ADF_Test(df, max_lags=10, variant=1)
189 res = adf.summary()
190
191 # Validate against statsmodels
192 ref = adfuller(df, maxlag=10, autolag='AIC', regression='c')
193 print(ref)

```

Listing 2: Test_stationarity.py — ADF and KPSS test using statsmodels

```

1 import numpy as np
2 import pandas as pd
3 from statsmodels.tsa.stattools import adfuller
4 from statsmodels.tsa.stattools import kpss
5
6 class Test_stationarity:
7     def __init__(self, time_series:pd.DataFrame, lag_max:int=None, regression_type:str="c", required_lag:str="AIC",
8                 regression_result:bool=False):
9         """Required I/P to test stationarity
10         Args:
11             time_series:pd.DataFrame: Time series or data series which is to analyze
12             lag_max:int: Number of max. lags can used, if None then default value is 12*(n_obs/100)^0.25
13             regression_type:str:{"c","ct","ctt","n"} Constant and trend order to include in regression.
14                 "c":constant only (default).
15                 "ct":constant and trend.
16                 "ctt":constant, and linear and quadratic trend.
17                 "n":no constant, no trend.
18             required_lag:str:{"AIC", "BIC", "t-stat", None}
19                 Method to use when automatically determining the lag length among values 0, 1, ..., maxlag.
20                 ~ If "AIC" (default) or "BIC", then the number of lags is chosen to minimize the corresponding
21                   information criterion.
22                 ~ "t-stat" based choice of maxlag. Starts with maxlag and drops a lag until the t-statistic on the last
23                   lag length is significant using a 5%-sized test.
24                 ~ If None, then the number of included lags is set to maxlag.
25             regression_result:bool=False: If True, the full regression results are returned. Default is False
26         O/P:
27             {adf_statistics,p-value,used_lags,n_observation,{cv_1%,cv_5%,cv_10%},ic_best(if autolag is not None),
28             res_store}"""
29
30         self.TS=time_series
31         self.maxlag=lag_max
32         self.regression_type=regression_type
33         self.required_lag=required_lag
34         self.regression_result=regression_result
35
36     def __repr__(self):
37         representation=str(f"class to test stationarity of Time series\nIP_TS:{self.TS.describe()}\nmaxlags:{self.maxlag
38                             },regression_type={self.regression_type}\nrequired_lag={self.required_lag}\nOP:")
39         return representation
40
41     def test_adf(self):
42         time_series=self.TS.squeeze()
43         test=adfuller(x=time_series,maxlag=self.maxlag,regression=self.regression_type,autolag=self.required_lag,
44                      regresults=self.regression_result)
45         summary={}
46         o_p=["adf_statistics","p-value","used_lags","n_observation","critical_values"]
47         for i,j in zip(test[0:],o_p[0:]):
48             if j=="critical_values":
49                 summary[j]={key:round(float(value),4) for key,value in i.items()}
50             else:
51                 summary[j]=round(float(i),4)
52         if len(test)>5:
53             summary["ic_best"]=round(float(test[5]),4)
54         if len(test)>6:
55             summary["result_store"]=round(float(test[6]),4)
56         print("Critical Values:")
57         for key, value in summary["critical_values"].items():
58             print(f"{key}: {value}")
59         return summary
60
61     def test_kpss(self):
62         time_series=self.TS.squeeze()
63         test=kpss(x=time_series,regression=self.regression_type)
64         summary={}
65         o_p=["kpss_statistics","p-values","lags","critical_values"]
66         for i,j in zip(test[0:],o_p[0:]):
67             if j=="critical_values":
68                 summary[j]={key:round(float(value),4) for key,value in i.items()}
69             else:
70                 summary[j]=round(float(i),4)
71         print("Critical Values:")
72         for key, value in summary["critical_values"].items():
73             print(f"{key}: {value}")
74         return summary

```

```
70 data_file=pd.read_csv(filepath_or_buffer=r"C:\Users\HP\OneDrive\Documents\Stock Projects\Vectron\data\storage\HAL.  
    NS_1d_2y.csv")  
71 df=pd.DataFrame(data=data_file["Close"])  
72 df=np.log(df/df.shift(1)).dropna()  
73 # print(df)  
74 test=Test_stationarity(time_series=df,lag_max=10,regression_type="ct")  
75 print(test.test_adf())  
76 print(test.test_kpss())
```